

Guía profesional de métodos de arreglos en JavaScript

Versión extendida — Nivel intermedio

Introducción

Los arreglos (arrays) en JavaScript son una de las estructuras más utilizadas para almacenar y manipular colecciones de datos.

JavaScript ofrece múltiples métodos para transformar, filtrar, buscar, modificar y recorrer los elementos de un arreglo.

Esta guía resume los métodos más usados, con sus sintaxis, propósito, ejemplos prácticos y comparaciones para que puedas recordarlos y aplicarlos con seguridad.

◆ SECCIÓN 1 — Métodos de Transformación

1. `.map()`

Descripción:

Crea un nuevo arreglo aplicando una función a cada elemento del arreglo original.

Sintaxis:

```
const nuevoArray = array.map((elemento, indice, array) => {  
  return /* nuevo valor */;  
});
```

Ejemplo:

```
const numeros = [1, 2, 3];  
const dobles = numeros.map(n => n * 2);  
console.log(dobles); // [2, 4, 6]
```

Notas:

- No modifica el arreglo original.

- Ideal para transformar datos.
-

2. `.filter()`

Descripción:

Crea un nuevo arreglo con los elementos que cumplen una condición.

Sintaxis:

```
const nuevoArray = array.filter((elemento, indice, array) => {  
  return /* condición booleana */;  
});
```

Ejemplo:

```
const edades = [15, 22, 30, 17];  
const mayores = edades.filter(e => e >= 18);  
console.log(mayores); // [22, 30]
```

Notas:

- Devuelve un arreglo (aunque sea de un solo elemento).
 - No modifica el arreglo original.
-

3. `.reduce()`

Descripción:

Reduce los elementos del arreglo a un único valor (numérico, string, objeto, etc.).

Sintaxis:

```
const resultado = array.reduce((acumulador, actual) => {  
  return acumulador + actual;  
}, valorInicial);
```

Ejemplo:

```
const numeros = [10, 20, 30];  
const suma = numeros.reduce((acum, num) => acum + num, 0);
```

```
console.log(suma); // 60
```

Notas:

- Muy útil para obtener totales, promedios o agrupar datos.
-

4. `.sort()`

Descripción:

Ordena los elementos del arreglo según una función de comparación.

Sintaxis:

```
array.sort((a, b) => a - b); // Ascendente numérico  
array.sort((a, b) => b - a); // Descendente numérico
```

Ejemplo:

```
const notas = [10, 5, 8];  
notas.sort((a, b) => a - b);  
console.log(notas); // [5, 8, 10]
```

Notas:

- **Modifica** el arreglo original.
 - Si no se pasa una función, ordena como *strings* (alfabéticamente).
-

◆ SECCIÓN 2 — Métodos de Búsqueda y Comparación

5. `.find()`

Descripción:

Devuelve el **primer elemento** que cumple una condición.

Ejemplo:

```
const alumnos = [  
  { nombre: "Ana", nota: 8 },
```

```
    { nombre: "Luis", nota: 5 }  
  ];  
const aprobado = alumnos.find(a => a.nota >= 6);  
console.log(aprobado); // { nombre: "Ana", nota: 8 }
```

6. `.some()`

Descripción:

Devuelve `true` si **al menos un elemento** cumple la condición.

Ejemplo:

```
const numeros = [1, 3, 5];  
console.log(numeros.some(n => n % 2 === 0)); // false
```

7. `.every()`

Descripción:

Devuelve `true` si **todos los elementos** cumplen la condición.

Ejemplo:

```
const notas = [7, 8, 9];  
console.log(notas.every(n => n >= 6)); // true
```

8. `.findIndex()`

Descripción:

Devuelve el **índice** del primer elemento que cumple una condición.

Ejemplo:

```
const edades = [15, 18, 21];  
console.log(edades.findIndex(e => e >= 18)); // 1
```

◆ SECCIÓN 3 — Métodos de Extracción y Modificación

9. `.slice()`

Descripción:

Devuelve una copia parcial del arreglo original.

Ejemplo:

```
const letras = ["a", "b", "c", "d"];
console.log(letras.slice(1, 3)); // ["b", "c"]
```

Notas:

- No modifica el arreglo original.
 - El segundo índice no se incluye.
-

10. `.splice()`

Descripción:

Agrega, reemplaza o elimina elementos en el arreglo.

Ejemplo:

```
const frutas = ["manzana", "banana", "uva"];
frutas.splice(1, 1, "pera");
console.log(frutas); // ["manzana", "pera", "uva"]
```

Notas:

- **Sí** modifica el arreglo original.
 - Sintaxis: `splice(indiceInicio, cantidadEliminar, elementosAgregar...)`
-

11. `.concat()`

Descripción:

Combina dos o más arreglos y devuelve uno nuevo.

Ejemplo:

```
const a = [1, 2];
const b = [3, 4];
const combinado = a.concat(b);
```

```
console.log(combinado); // [1, 2, 3, 4]
```

12. .includes()

Descripción:

Verifica si un elemento existe en el arreglo.

Ejemplo:

```
const colores = ["rojo", "verde", "azul"];  
console.log(colores.includes("verde")); // true
```

◆ SECCIÓN 4 — Comparaciones Visuales

✂ map() vs filter() vs reduce()

Método	Qué hace	Devuelve	Modifica original
.map()	Transforma cada elemento	Nuevo arreglo	✗ No
.filter() ()	Filtra elementos que cumplen condición	Nuevo arreglo	✗ No
.reduce() ()	Acumula en un solo valor	Cualquier tipo	✗ No

Ejemplo comparativo:

```
const numeros = [1, 2, 3, 4, 5];  
console.log(numeros.map(n => n * 2)); // [2,4,6,8,10]  
console.log(numeros.filter(n => n > 3)); // [4,5]  
console.log(numeros.reduce((a,b) => a+b)); // 15
```

✂ slice() vs splice()

Método	Modifica el original	Qué devuelve	Uso común
.slice() ()	✗ No	Copia parcial	Clonar o mostrar porciones

`.splice`  Sí
()

Elementos
removidos

Insertar o borrar datos

SECCIÓN 5 — Notas Profesionales y Buenas Prácticas

-  Evita modificar el arreglo original salvo que sea estrictamente necesario.
-  Usa `const` para declarar arreglos que no serán reasignados.
-  Combina `map` y `filter` para transformaciones más limpias:

```
const activos = usuarios.filter(u => u.activo).map(u => u.nombre);
```

-  Cuando uses `reduce()`, siempre define un valor inicial para evitar errores con arreglos vacíos.
-  Evita anidar muchos métodos seguidos sin claridad; usa saltos de línea para legibilidad.

Profundización sobre los parámetros

Los tres parámetros del `.map()`

La función que le pasas al `.map()` puede recibir **hasta tres argumentos opcionales**:

```
array.map((elemento, indice, arrayOriginal) => { ... })
```

elemento

 Es el **valor actual** del elemento que está recorriendo `.map()` en ese momento. Este es el parámetro que tú siempre usas (por ejemplo, `est` o `n` o `item`).

Ejemplo:

```
const numeros = [10, 20, 30];
numeros.map(num => console.log(num));
// Imprime 10, luego 20, luego 30
```

2 índice

→ Es el **número de posición** (0, 1, 2, ...) del elemento dentro del arreglo. Esto es útil cuando quieres mostrar o usar la posición de cada elemento.

Ejemplo:

```
const frutas = ["Manzana", "Pera", "Uva"];
frutas.map((fruta, i) => console.log(`${i}: ${fruta}`));
// Imprime:
// 0: Manzana
// 1: Pera
// 2: Uva
```

3 arrayOriginal

→ Es una **referencia al arreglo completo** sobre el que estás ejecutando `.map()`. Esto sirve si necesitas comparar el elemento actual con otros o hacer cálculos usando todo el arreglo.

Ejemplo:

```
const numeros = [2, 4, 6];
const doblesYTotal = numeros.map((num, i, arr) => {
  return `${num * 2} de ${arr.length} elementos`;
});
console.log(doblesYTotal);
// ["4 de 3 elementos", "8 de 3 elementos", "12 de 3 elementos"]
```

✓ En resumen:

Parámetro	Qué representa	Caso común
<code>elemento</code>	Valor actual	Transformar cada dato
<code>índice</code>	Posición en el arreglo	Numerar o acceder por posición
<code>array</code>	El arreglo completo	Calcular algo con el total

 Nota:

No es obligatorio usar los tres.

Puedes usar solo los que necesites, por ejemplo:

```
array.map((elemento) => { ... })           // Más común
array.map((elemento, indice) => { ... })    // Si necesitas la
posición
array.map((elemento, indice, array) => { ... }) // Si necesitas
acceso total
```

Resumen completo: Usos profesionales del método `reduce()`

El método `reduce()` es uno de los más poderosos y flexibles de JavaScript.

Sirve para transformar un array en *cualquier otra cosa*:

- un número
- un string
- un objeto
- otro array
- un array agrupado
- un resumen de datos
- una estructura completamente nueva

1. Sintaxis general

```
array.reduce((acumulador, elementoActual, indice, arrayOriginal) =>
{
    // lógica
```

```
    return nuevoAcumulador;  
  }, valorInicial);
```

Parámetros:

- **acumulador:** el valor que se va construyendo en cada iteración.
 - **elementoActual:** el valor del elemento que se está iterando.
 - **índice (opcional):** posición del elemento.
 - **arrayOriginal (opcional):** el array entero.
 - **valorInicial:** valor desde el que parte el acumulador.
-



2. Casos de uso principales

A) Sumar valores (el uso más común)

```
const nums = [5, 10, 15];  
const suma = nums.reduce((a, n) => a + n, 0);  
// 30
```

B) Transformar un array en un objeto (contar ocurrencias)

```
const letras = ["a", "b", "a", "c", "b", "a"];  
  
const frecuencias = letras.reduce((acum, letra) => {  
  acum[letra] = (acum[letra] || 0) + 1;  
  return acum;  
}, {});  
  
console.log(frecuencias);  
// { a: 3, b: 2, c: 1 }
```

Cuándo usarlo:

Cuando quieres convertir un array en un objeto basado en conteos, estadísticas o índices.

C) Transformar un array en otro array

```
const nums = [1, 2, 3];

const duplicados = nums.reduce((acum, n) => {
  acum.push(n * 2);
  return acum;
}, []);

console.log(duplicados);
// [2, 4, 6]
```

 Esto es similar a `map()`, pero más flexible si quieres hacer lógica adicional.

D) Agrupar datos por categorías

```
const productos = [
  { nombre: "Laptop", categoria: "tecnologia" },
  { nombre: "Mouse", categoria: "tecnologia" },
  { nombre: "Pan", categoria: "comida" }
];

const agrupados = productos.reduce((acum, p) => {
  if (!acum[p.categoria]) acum[p.categoria] = [];
  acum[p.categoria].push(p);
  return acum;
}, {});

console.log(agrupados);
/*
{
  tecnologia: [
    { nombre: "Laptop", categoria: "tecnologia" },
    { nombre: "Mouse", categoria: "tecnologia" }
  ],
  comida: [
```

```
    { nombre: "Pan", categoria: "comida" }  
  ]  
}  
*/
```

Cuándo usarlo:

Cuando quieres reorganizar datos para tener grupos por categoría, fecha, tipo, etc.

E) Convertir un array en un solo string

```
const palabras = ["hola", "mundo"];  
  
const frase = palabras.reduce((a, p) => a + " " + p);  
console.log(frase);  
// "hola mundo"
```

F) Buscar el máximo o el mínimo

```
const nums = [10, 3, 25, 6];  
  
const max = nums.reduce((a, n) => n > a ? n : a);  
// 25
```

G) Aplanar arrays anidados

```
const nested = [[1, 2], [3, 4], [5]];  
  
const plano = nested.reduce((a, arr) => a.concat(arr), []);  
// [1, 2, 3, 4, 5]
```

3. Diferencia entre los casos B (contar) y D (agrupar)

Característica	Caso B: Contar	Caso D: Agrupar
Devuelve	Objeto con números	Objeto con arrays
Ejemplo	<code>{a: 3, b: 2}</code>	<code>{tecnologia: [...], comida: [...]}</code>
Objetivo	Contar repetidos	Organizar datos
Ideal para	Estadísticas	Tablas, catálogos, dashboards
Estructura	<code>acum[clave] = número</code>	<code>acum[clave] = [] → push()</code>

4. Patrones comunes con reduce()

Patrón: contador

```
acum[key] = (acum[key] || 0) + 1;
```

Patrón: agrupador

```
if (!acum[key]) acum[key] = [];
acum[key].push(item);
```

Patrón: acumulación numérica

```
return acum + valor;
```

5. Errores comunes

- ✓ Olvidar el valor inicial
- ✓ No retornar el acumulador
- ✓ Usar mal las llaves `{...}` sin return
- ✓ Confundir "contar" con "agrupar"